

Министерство науки и высшего образования Российской Федерации
федеральное государственное автономное образовательное учреждение
высшего образования «Национальный исследовательский университет
ИТМО»

Факультет программной инженерии и компьютерной техники

Встроенные системы

Мастер-класс №3
Вариант: 2

Выполнили:

Вильданов Ильнур Наилевич
Бондарев Алексей Михайлович
Группа №Р3312

Проверил:

Гончаров Алексей Андреевич

Санкт-Петербург
2025 г.

Текст задания

Требуется реализовать игру «Тетрис» на микроконтроллере STM32 с применением ОС реального времени FreeRTOS и монохромного OLED-дисплея.

Выполнение

В ходе выполнения были:

- настроены GPIO, I2C, UART и RNG;
- подключен и инициализирован OLED-дисплей;
- создана отдельная задача FreeRTOS для игры;
- организован ввод с кнопок через очередь;
- реализована игровая логика (движение, повороты, коллизии, удаление линий, очки, ускорение);
- выполнена отрисовка поля и фигур на дисплее.

Структура проекта

Файл	Назначение
main.c	Инициализация HAL/периферии, запуск FreeRTOS
tetris_task.c	Логика и отрисовка игры
cmsis_os2.c / freertos.c	Задачи/очереди RTOS
oled.h / oled.c	Вывод на дисплей
app_queues.h	Объявления очередей (в т.ч. myKbdQueueHandle)
rng.h / rng.c	Работа с RNG

Описание реализации

Основной модуль main.c

main.c выполняет стандартную инициализацию: HAL, тактирование, периферию, OLED и запуск планировщика FreeRTOS.

```
int main(void)
{
    HAL_Init();
    SystemClock_Config();

    MX_GPIO_Init();
    MX_I2C1_Init();
    MX_USART6_UART_Init();
    MX_RNG_Init();

    oled_Init();
    oled_Fill(Black);
    oled_UpdateScreen();

    MX_FREERTOS_Init();
    osKernelStart();
}
```

```
while (1){}
}
```

Задача Tetris_Task (tetris_task.c)

Задача реализует игровую логику и отрисовку в одном цикле. Поле - 10×20, фигуры задаются битовыми масками 4×4 и имеют 4 ориентации.

Параметры поля

```
#define BOARD_W 10
#define BOARD_H 20
#define CELL_PX 3
#define CELL_FILL 2
#define FIELD_X 2
#define FIELD_Y 2
```

Состояние поля хранится в массиве board (0 – пусто, 1 – занято):

```
static uint8_t board[BOARD_H][BOARD_W];
```

Описание фигур

```
typedef enum {
    PIECE_I = 0,
    PIECE_J,
    PIECE_L,
    PIECE_O,
    PIECE_S,
    PIECE_T,
    PIECE_Z,
    PIECE_COUNT
} piece_t;
```

Фигуры заданы набором масок (4 поворота):

```
static const uint16_t PIECES[PIECE_COUNT][4] = {
    { 0x0F00, 0x2222, 0x00F0, 0x4444 }, /* I */
    { 0x8E00, 0x6440, 0x0E20, 0x44C0 }, /* J */
    { 0x2E00, 0x4460, 0x0E80, 0xC440 }, /* L */
    { 0x6600, 0x6600, 0x6600, 0x6600 }, /* O */
    { 0x6C00, 0x4620, 0x06C0, 0x8C40 }, /* S */
    { 0x0E40, 0x4C40, 0x4E00, 0x4640 }, /* T */
    { 0xC600, 0x2640, 0x0C60, 0x4C80 } /* Z */
};
```

Текущая фигура хранится в структуре:

```
typedef struct {
    uint8_t type;
    uint8_t rot;
    int8_t x;
    int8_t y;
} active_piece_t;

static active_piece_t cur;
```

RNG и появление новой фигуры

Выбор следующей фигуры выполняется через аппаратный RNG (с запасным вариантом).

```
static uint32_t hw_rand_u32(void)
{
    uint32_t v = 0;

    if (HAL_RNG_GenerateRandomNumber(&hrng, &v) == HAL_OK) return v;
    return osKernelSysTick() ^ 0xA5A5A5A5u;
}
```

Коллизии, движение и поворот

Коллизии проверяются по границам поля и по занятым клеткам board.

```
static uint8_t collides(uint8_t type, uint8_t rot, int8_t px, int8_t py) {
    uint16_t m = PIECES[type][rot & 3u];
    for (uint8_t y = 0; y < 4; y++) {
        for (uint8_t x = 0; x < 4; x++) {
            if (!mask_cell(m, x, y)) continue;
            int16_t bx = (int16_t)px + (int16_t)x;
            int16_t by = (int16_t)py + (int16_t)y;
            if (bx < 0 || bx >= BOARD_W) return 1;
            if (by < 0 || by >= BOARD_H) return 1;
            if (board[by][bx]) return 1;
        }
    }

    return 0;
}
```

Движение и поворот выполняются только если новая позиция валидна.

Очистка линий и очки

Удаляются полностью заполненные линии, выше лежащие строки сдвигаются вниз. Очки начисляются: 100/300/500/800 за 1/2/3/4 линии за раз.

```
static uint8_t clear_lines(void) {
    uint8_t cleared = 0;
    for (int16_t y = BOARD_H - 1; y >= 0; y--) {
        uint8_t full = 1;
        for (uint8_t x = 0; x < BOARD_W; x++) {
            if (!board[y][x]) { full = 0; break; }
        }
        if (full) {
            cleared++;
            for (int16_t yy = y; yy > 0; yy--) memcpy(board[yy], board[yy - 1], BOARD_W);
            memset(board[0], 0, BOARD_W);
            y++;
        }
    }
    if (cleared) {
        lines_total += cleared;
        switch (cleared) {
```

```

        case 1: score += 100; break;
        case 2: score += 300; break;
        case 3: score += 500; break;
        default: score += 800; break;
    }
    set_fall_speed_from_level();
}
return cleared;
}

```

Отрисовка

Поле и текущая фигура рисуются в буфер дисплея, затем вызывается oled_UpdateScreen(). Дополнительно выводится счет.

Основной цикл и управление

Ввод считывается из очереди. Экран перерисовывается только при изменениях (флаг dirty).

```

void Tetris_Task(void const *argument) {
    (void)argument;

    (void)oled_Init();
    board_clear();
    score = 0;
    lines_total = 0;
    set_fall_speed_from_level();
    spawn_piece();
    render();

    for (;;) {
        uint32_t now = rtos_tick();
        uint8_t dirty = 0;

        uint8_t key;

        while (queue_try_rcv_u8(&key)) {
            if (key == 4) { if (try_move(-1, 0)) dirty = 1; }
            else if (key == 6) { if (try_move(1, 0)) dirty = 1; }
            else if (key == 2) { if (try_rotate()) dirty = 1; }
            else if (key == 5) {
                if (!try_move(0, 1)) { lock_piece(); clear_lines(); spawn_piece(); }
                last_fall_tick = now;
                dirty = 1;
            } else if (key == 8) { hard_drop(); dirty = 1; }
        }

        if ((now - last_fall_tick) >= fall_ms) {
            last_fall_tick = now;
            if (!try_move(0, 1)) { lock_piece(); clear_lines(); spawn_piece(); }
            dirty = 1;
        }
    }
}

```

```
if (dirty) render();  
osDelay(LOOP_DELAY_MS);  
}
```

Коды управления (из очереди): 4 - влево, 6 - вправо, 2 - поворот, 5 - мягкое падение, 8 - жесткое падение.

Особенности использования FreeRTOS

Время задается системным тиком ОС (osKernelSysTick()/osKernelGetTickCount()), обернутым в rtoс_tick(). Ввод от кнопок передается в задачу игры через очередь myKbdQueueHandle.

Чтение из очереди - неблокирующее, что позволяет быстро обрабатывать серию нажатий.

```
static inline uint8_t queue_try_rcv_u8(uint8_t *out) {  
    osEvent ev = osMessageGet(myKbdQueueHandle, 0);  
  
    if (ev.status == osEventMessage) {  
        *out = (uint8_t)(ev.value.v);  
        return 1;  
    }  
  
    return 0;  
}
```

Ссылка на реализацию

<https://github.com/h3ywh4t/embedded-systems-lab-3>

Вывод

Реализована игра «Тетрис» на STM32 с использованием FreeRTOS и OLED-дисплея. В ходе работы были освоены: инициализация периферии STM32 (GPIO, I2C, UART, RNG); базовые механизмы RTOS (задачи, очереди, планировщик); реализация игровой логики (коллизии, фиксация фигур, очистка линий, подсчет очков, ускорение); формирование графического вывода на монохромный дисплей.